Practical No. 3

Structured Text Generation

Subject	Generative Artificial Intelligence
Name / Roll No.	_____
Date	_____

Aim

To build an application that takes input from the user and uses Generative AI to produce structured output (JSON, a question bank and a table).

Software Requirements

- **Operating System:** Windows 10/11, Linux (Ubuntu) or macOS
- **Programming Language:** Python 3.9 or above
- **IDE / Platform:** Google Colab, Jupyter Notebook or VS Code
- **Libraries:** `groq` (install with `pip install groq`) and `json` (built into Python)
- **API:** Groq API (free tier) with an API key from console.groq.com
- **Web Browser:** Google Chrome, Firefox or Edge (needed for Colab and for getting the API key)

Hardware Requirements

- **Computer:** Desktop PC or laptop
- **Processor:** Intel Core i3 (or equivalent) or higher
- **RAM:** 4 GB minimum (8 GB recommended)
- **Storage:** At least 1 GB of free disk space
- **Network:** Active internet connection (the model runs on the Groq cloud, so a GPU is not required)
- **Peripherals:** Monitor, keyboard and mouse

Theory

Structured text generation: An LLM normally replies in free-form text. Structured generation makes the model reply in a fixed, machine-readable format such as JSON, a table, a list or a report template. Programs can then read the output directly.

JSON (JavaScript Object Notation): A lightweight format that stores data as key-value pairs and lists. Python converts JSON text into a dictionary using `json.loads()`.

How structure is achieved:

- **Prompt instructions:** tell the model the exact format and show an example.
- **JSON mode:** the API setting `response_format={"type": "json_object"}` forces valid JSON output.
- **Parsing:** `json.loads()` converts the reply text into a Python object, which the program can print as a table or store.

Applications: question-bank generation, resume parsing, data extraction from text, report generation, form filling, and feeding model output into databases and other software.

Challenge: Models can occasionally return broken or wrong-format output, so the output must be parsed and validated safely.

Algorithm

1. Start.
2. Import the libraries and create the Groq client with the API key.
3. Define a function `ask(prompt)` that sends a prompt to the model (JSON mode on) and returns the reply.
4. Take the topic from the user.
5. Build a prompt asking for 3 multiple-choice questions in a fixed JSON format.
6. Send the prompt and receive the JSON text.
7. Convert the JSON text into a Python dictionary using `json.loads()`.
8. Loop through the questions and display them as a formatted question bank.
9. Display the answers in a table.
10. Stop.

Program Code

```
import json
from groq import Groq

client = Groq(api_key="YOUR_GROQ_KEY")

def ask(prompt):
    reply = client.chat.completions.create(
        model="openai/gpt-oss-120b",
        messages=[{"role": "user", "content": prompt}],
        response_format={"type": "json_object"})
    return reply.choices[0].message.content

# 1. Take input
topic = input("Enter a topic: ")

# 2. Ask for structured output
prompt = f"""Create 3 multiple choice questions on {topic}.
Return only JSON in this format:
{{"questions": [
  {{"question": "...", "options": ["A. ...", "B. ...", "C. ...", "D. ..."], "answer": "A"}}
]}}"""

text = ask(prompt)
print("\nRAW JSON OUTPUT:\n", text)

# 3. Convert JSON text to Python dictionary
data = json.loads(text)

# 4. Display as question bank
print("\nQUESTION BANK ON", topic.upper())
for i, q in enumerate(data["questions"], 1):
    print(f"\nQ{i}. {q['question']}")
    for opt in q["options"]:
        print("   ", opt)

# 5. Display answers as table
print("\nANSWER KEY")
print("Q.No | Answer")
print("-----|-------")
for i, q in enumerate(data["questions"], 1):
    print(f"{i:<4} | {q['answer']}")
```

Note: Get a free key from console.groq.com/keys and replace `YOUR_GROQ_KEY`. In Colab, run `!pip install groq` in a separate cell first. If the model name gives a 404 error, list the available models with `client.models.list()` and use a current one.

Output

Sample output (the questions will differ on every run):

```
Enter a topic: Machine Learning

RAW JSON OUTPUT:
{"questions": [{"question": "What is supervised learning?",
 "options": ["A. Learning without data", "B. Learning from labeled data",
 "C. Learning from rewards only", "D. Learning without a model"],
 "answer": "B"}, ...]}

QUESTION BANK ON MACHINE LEARNING

Q1. What is supervised learning?
    A. Learning without data
    B. Learning from labeled data
    C. Learning from rewards only
    D. Learning without a model

Q2. Which of these is a classification algorithm?
    A. Linear Regression
    B. K-Means
    C. Decision Tree
    D. PCA

Q3. What is overfitting?
    A. Model fits training data too closely
    B. Model is too small
    C. Model has no parameters
    D. Model trains too fast

ANSWER KEY
Q.No | Answer
-----|-------
1    | B
2    | C
3    | A
```

Paste a screenshot of your own program output here in the journal.

Conclusion

A Python application was built that uses Generative AI to create structured output from user input. The model returned valid JSON, which was parsed with `json.loads()` and displayed as a question bank and an answer table. Structured generation makes LLM output directly usable by other programs.

Viva Questions

No.	Question	Answer
1	What is structured output?	Model output in a fixed machine-readable format such as JSON or a table, instead of free text.
2	What is JSON?	A text format that stores data as key-value pairs and lists.
3	What does json.loads() do?	It converts a JSON string into a Python dictionary or list.
4	What is JSON mode?	An API setting that forces the model to return valid JSON.
5	Why can parsing fail?	The model may return broken JSON or extra text, so use JSON mode and a clear prompt.
6	Where is this used?	Question banks, data extraction, report generation, and feeding model output into apps and databases.